

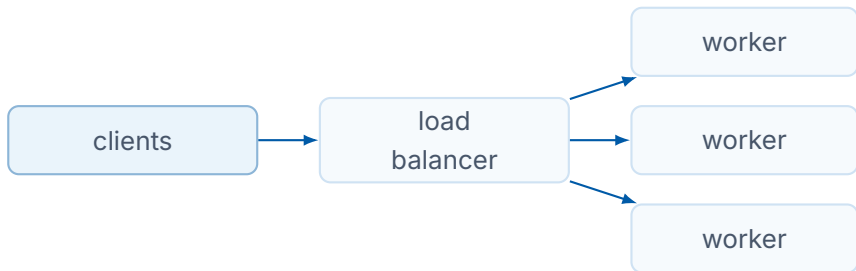


Scalable Distributed Computing - 2026/27

Lecture 2 - Architecture and Sources of Scalability Constraints

Patrizio Dazzi `patrizio.dazzi@unipi.it`

Where we ended last time



Horizontal scaling is easiest when requests are independent and workers are interchangeable.



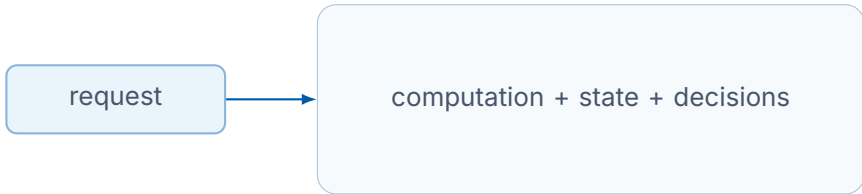
What changes when we remove those assumptions?

State, decisions and dependencies must now be
placed somewhere.

1. Decomposition creates boundaries

Before we can distribute a system, we must decide what its parts are

A system before decomposition



Everything is local because there is only one boundary.

When decomposition becomes a scalability problem



Scaling the workload was only part of the problem.

The structure of the software itself had to scale.

Where should the boundaries be?

Uber Engineering, *Service-Oriented Architecture: Scaling the Uber Engineering Codebase As We Grow*, 2015.

The decomposition criterion matters

Parnas' central lesson

Systems can be decomposed in different ways; a decomposition should be chosen according to the design decisions we want each module to hide and localize.

D. L. Parnas, *On the Criteria To Be Used in Decomposing Systems into Modules*, CACM 1972.

The decomposition criterion matters

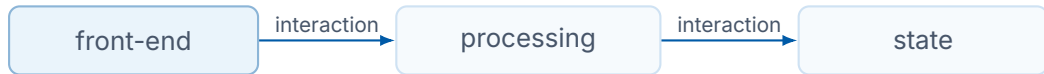
Parnas' central lesson

Systems can be decomposed in different ways; a decomposition should be chosen according to the design decisions we want each module to hide and localize.

For distributed systems, decomposition also determines which dependencies may cross machine boundaries.

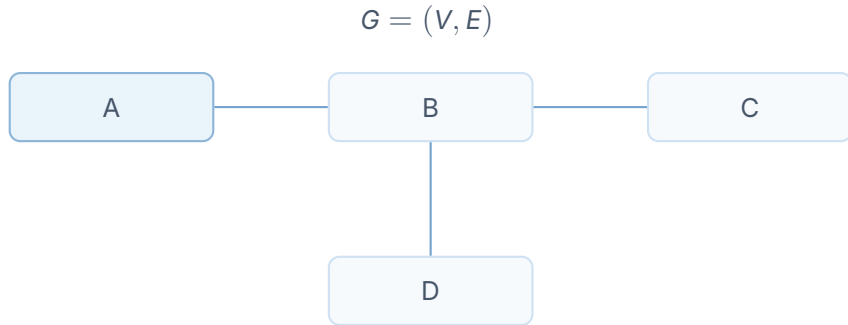
D. L. Parnas, On the Criteria To Be Used in Decomposing Systems into Modules, CACM 1972.

A decomposition changes what can become remote



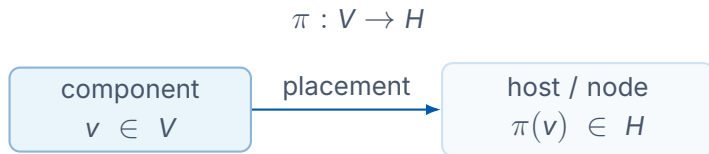
A local function call can become a protocol interaction; a local memory access can become remote state access.

Architecture as a dependency graph



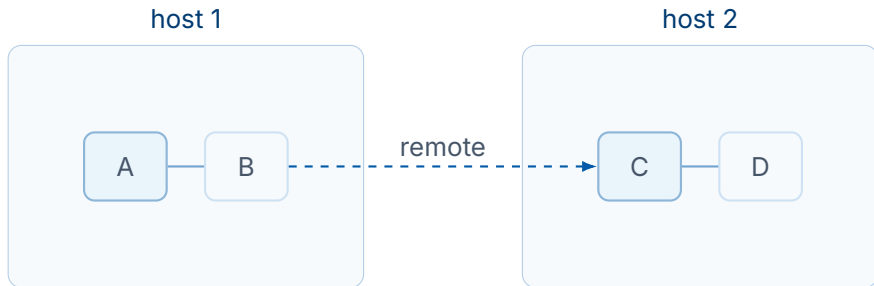
Vertices model components; edges model dependencies that require information exchange.

Placement is a second decision



Decomposition chooses the parts; placement chooses where they live.

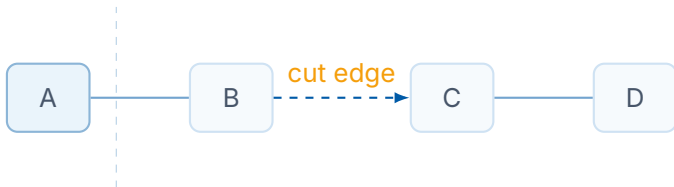
A dependency may be local or remote



A boundary turns some dependencies into network communication.

The cut induced by a placement

$$E_{\text{cut}}(\pi) = \{(u, v) \in E : \pi(u) \neq \pi(v)\}$$



A first communication model

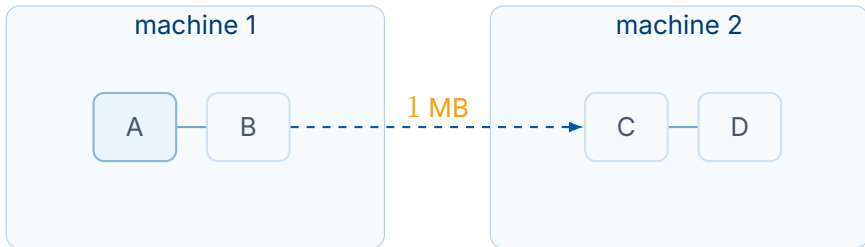
If each dependency e transfers w_e bytes per unit of work, define

$$C_{\text{edge}}(\pi) = \sum_{e \in E_{\text{cut}}(\pi)} w_e.$$

The same dependency graph can induce different communication volumes under different placements.

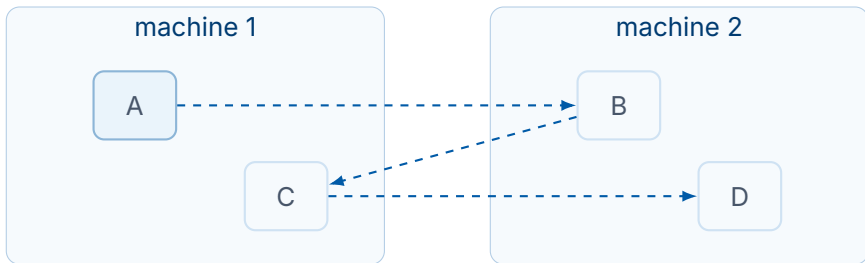
This is a deliberately simple weighted-cut model, used here as an architectural abstraction.

Same graph: placement A



$$C_{\text{edge}}(\pi_A) = 1 \text{ MB/iteration.}$$

Same graph: placement B

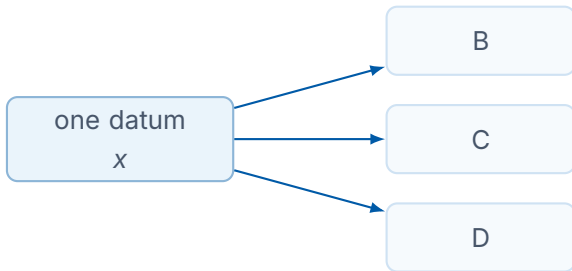


$$C_{\text{edge}}(\pi_B) = 3 \text{ MB/iteration.}$$

Placement changes communication

**Same computation, same
machines, same logical
dependencies but
different placement,
different communication.**

But edge cut is a simplified communication metric, not a complete communication model



Three graph edges may represent copies of the *same* datum, not three independent communication objects. Not necessarily always "activated".

See Hendrickson and Kolda, *Graph partitioning models for parallel computing*, Parallel Computing 2000.

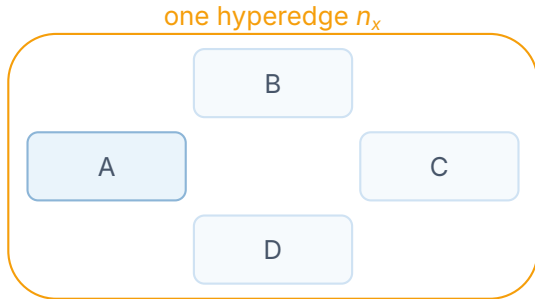
2. Hypergraphs: a richer model of shared dependencies

A a better language for reasoning about placement

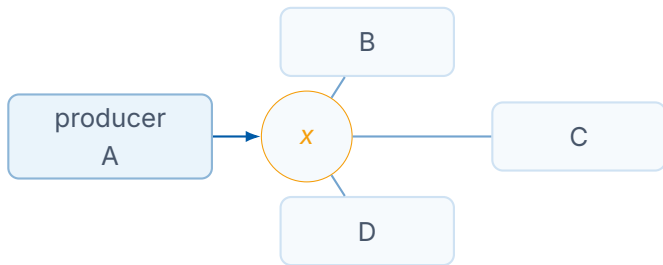
What is a hypergraph?

Definition

A hypergraph is $H = (V, \mathcal{N})$, where each hyperedge (or net) $n \in \mathcal{N}$ may connect an arbitrary subset of vertices, not just two.



One hyperedge can represent one shared datum



The hyperedge groups all producer/consumer dependencies that concern the same communication object.

Connectivity of a hyperedge

For a partition $\Pi = \{V_1, \dots, V_p\}$, define

$$\lambda_n(\Pi) = |\{i : n \cap V_i \neq \emptyset\}|.$$

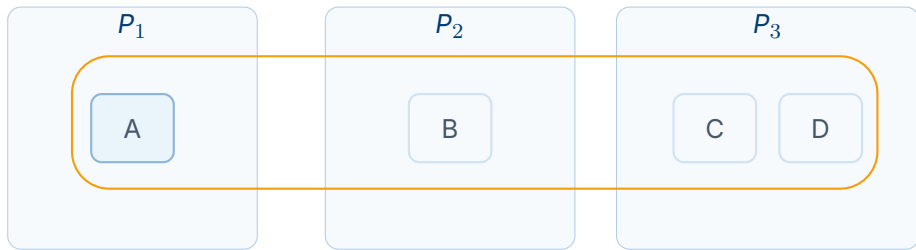
λ_n is the number of partitions touched by hyperedge n .

If a datum touches two partitions



$$\lambda_x = 2 \quad \Rightarrow \quad \lambda_x - 1 = 1.$$

If the same datum touches three partitions



$$\lambda_x = 3 \quad \Rightarrow \quad \lambda_x - 1 = 2.$$

The connectivity-1 communication metric

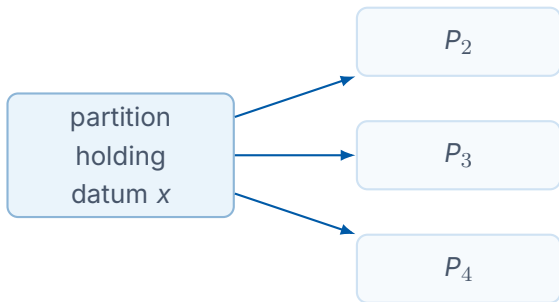
If hyperedge n represents c_n units of data, the standard metric is

$$C(\Pi) = \sum_{n \in \mathcal{N}} c_n (\lambda_n(\Pi) - 1)$$

Under the corresponding hypergraph communication model, this metric captures total communication volume.

Çatalyürek and Aykanat, IEEE TPDS 1999; Hendrickson and Kolda, Parallel Computing 2000.

Why $\lambda_n - 1$ has a physical meaning



If x is required on λ_x partitions, it must reach the other $\lambda_x - 1$ partitions.

A small weighted example

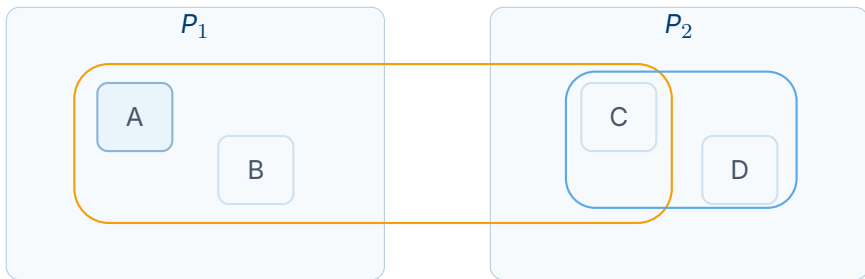
Consider two shared dependencies:

$$n_x = \{A, B, C\}, \quad c_x = 10 \text{ MB}$$

$$n_y = \{C, D\}, \quad c_y = 1 \text{ MB.}$$

We will compare two equally balanced placements on two partitions.

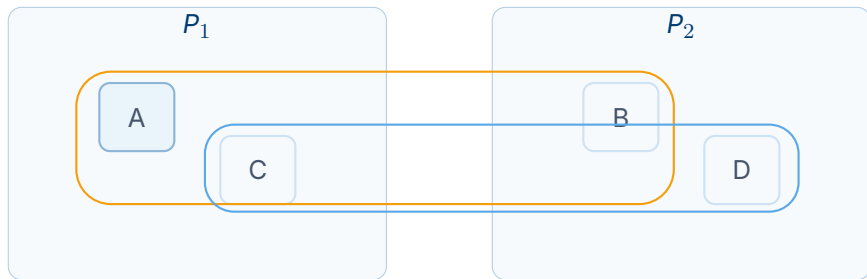
Placement A



$$\lambda_x = 2, \quad \lambda_y = 1$$

$$C(\Pi_A) = 10(2 - 1) + 1(1 - 1) = 10 \text{ MB.}$$

Placement B



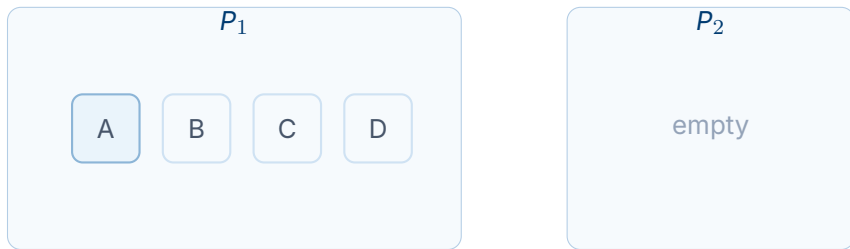
$$\lambda_x = 2, \quad \lambda_y = 2$$

$$C(\Pi_B) = 10(2 - 1) + 1(2 - 1) = 11 \text{ MB.}$$

What changed?

**Not the computation. Not
the number of nodes. Not
the balance. Only the
placement of dependent
components.**

If communication were the only objective...



$$c(\Pi) = 0.$$

Perfect communication locality

But communication is not the only objective

If we only minimize communication, the best solution is trivial:

$$V_1 = V, \quad V_2 = \dots = V_p = \emptyset$$

$$C(\Pi) = 0$$

But then we have not distributed the work.

We also need to balance the load.

Let $W(V)$ be the total computational work associated with all vertices.

With p partitions, the **ideal load per partition** is

$$\boxed{\frac{W(V)}{p}}$$

Allowing a small imbalance

Perfect balance may be too restrictive. We therefore allow each partition to exceed the ideal load by a small fraction ϵ :

$$W(V_i) \leq (1 + \epsilon) \frac{W(V)}{p}$$

where $W(V_i)$ indicates the work assigned to partition i , ϵ represents the tolerated relative imbalance. For example, if $W(V) = 100$, $p = 4$, $\epsilon = 0.1$ then

$$\frac{W(V)}{p} = 25$$

and every partition must satisfy

$$W(V_i) \leq 27.5.$$

Thus the real problem is constrained

$$\min_{\Pi} \sum_n c_n (\lambda_n - 1)$$

subject to a balance constraint such as

$$w(V_i) \leq (1 + \epsilon) \frac{w(V)}{p}.$$

Locality and balance pull in different directions.

Balanced graph/hypergraph partitioning is a standard combinatorial optimization problem; practical systems rely on heuristics and multilevel methods.

What this result is for in this course



The formalism gives us a disciplined way to ask whether a decomposition can be placed so that expensive dependencies remain sufficiently local.

A useful architectural diagnostic

If every balanced placement of a decomposition produces heavy cross-boundary communication, the decomposition itself may deserve reconsideration.

Communication volume is not communication time

Two placements can have the same $C(\Pi)$ and still behave differently because of:

- message count and batching;
- per-node traffic concentration;
- synchronization structure;
- congestion and topology;
- latency vs bandwidth costs;
- overlap with computation.

Hendrickson and Kolda 2000 discuss limitations of graph metrics and richer communication objectives.

A simple latency-bandwidth refinement

For a message carrying m bytes, a common first-order model is

$$T_{\text{msg}} \approx \alpha + \beta m,$$

where α captures fixed startup/latency cost and β the marginal cost per byte. Thus a schematic placement cost is

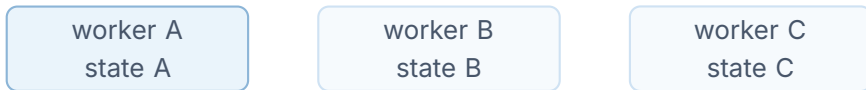
$$T_{\text{comm}}(\Pi) \approx \sum_{e \in E_{\text{cut}}} (\alpha + \beta w_e).$$

We will study richer communication and scalability costs in Lecture 3; this model is only a bridge.

3. Where does the state live?

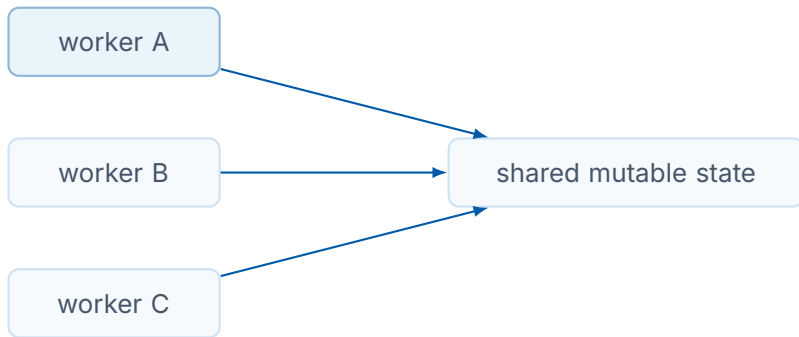
State placement changes which dependencies can remain local

The easy case: private state



State accesses remain local when ownership and computation coincide.

Shared mutable state creates a common dependency



Scaling the workers does **not** automatically scale the state dependency they share.

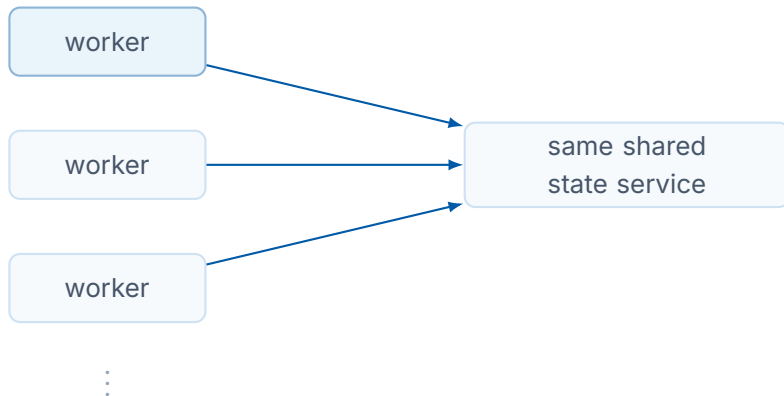
A tiny load model for shared state

Suppose requests arrive at rate λ and each request performs on average r state accesses.

$$\lambda_s = r\lambda$$

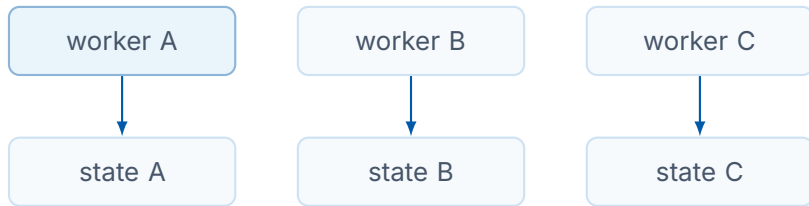
The offered load to the state tier is determined by workload and access intensity, not by the number of workers alone.

More workers do not imply more state capacity



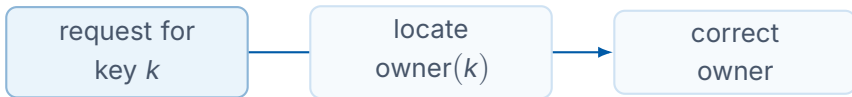
$N_{\text{workers}} \uparrow \not\Rightarrow \text{state capacity} \uparrow$

State ownership changes the dependency structure



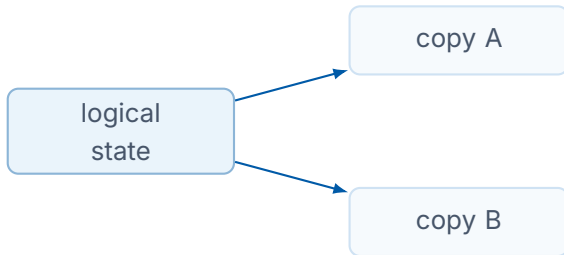
Locality improves when computation is placed with the state it owns.

But ownership creates a new architectural question



Localizing state replaces one dependency with another:
knowing where the state is.

Copies of state create another question



What must happen when more than one copy can change?
We postpone the answer to replication and consistency!!!

Shared-nothing as a historical design point

Stonebraker's shared-nothing idea

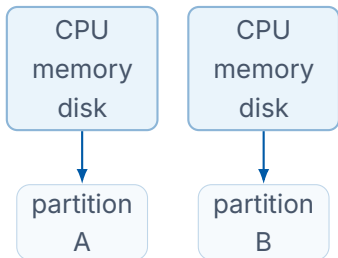
Processors own private memory and disks; communication happens through an interconnection network rather than shared memory or shared disk.

The architectural intuition is to avoid globally shared hardware resources and partition state/processing ownership.

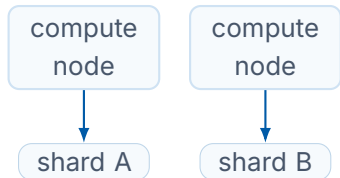
M. Stonebraker, *The Case for Shared Nothing*, Database Engineering Bulletin, 1986.

Shared-nothing did not disappear

Shared-nothing principle



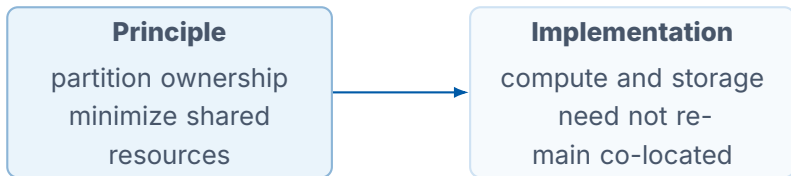
Modern sharded service



The enduring idea is **local ownership with minimal shared resources**: partition the state, route work to its owner, and exploit local processing.

Stonebraker, Database Engineering Bulletin 1986; see also contemporary database-sharding architectures.

The principle survives; the physical architecture may change



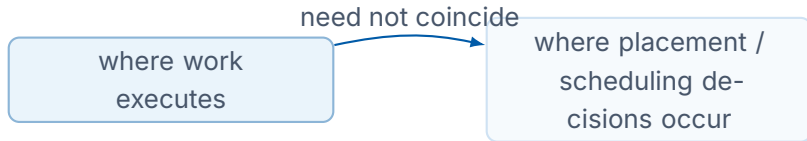
Modern systems may **retain, reinterpret, or deliberately relax** the shared-nothing principle.

This distinction will return when we study **data locality and storage/compute disaggregation.**

4. Where are decisions made?

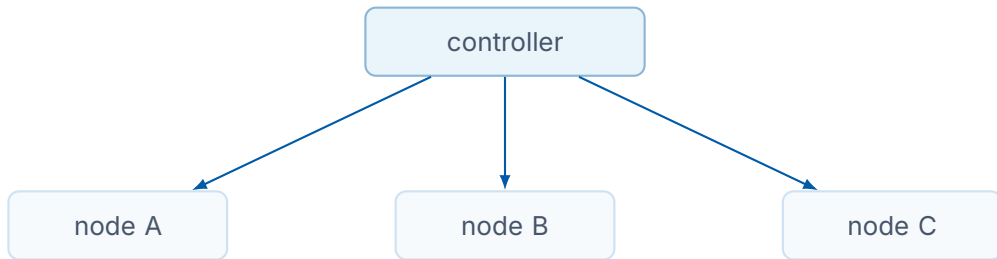
Execution and control need not live in the same place

Execution locus $\neq$ decision locus



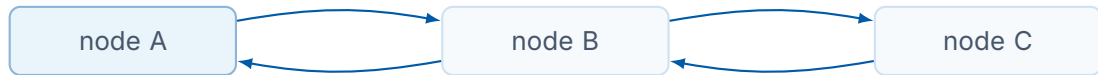
Separating execution from decision-making creates a control structure.

Centralized decision authority



Information and authority are localized; the controller becomes a dependency for decisions.

Distributed decision-making



Decisions must now be made from information that is itself distributed and potentially delayed.

Control placement is a trade-off, not a hierarchy

Centralized authority	Distributed authority
simpler global decision point information must flow inward control dependency is explicit	avoids one global decision point information remains dispersed coordination burden is distributed

The right placement depends on the decision being made
and its required scope.

5. How do components interact?

Interaction structure determines temporal dependencies

A direct synchronous dependency



A cannot complete this step until B has participated.

Sequential dependency adds to the critical path

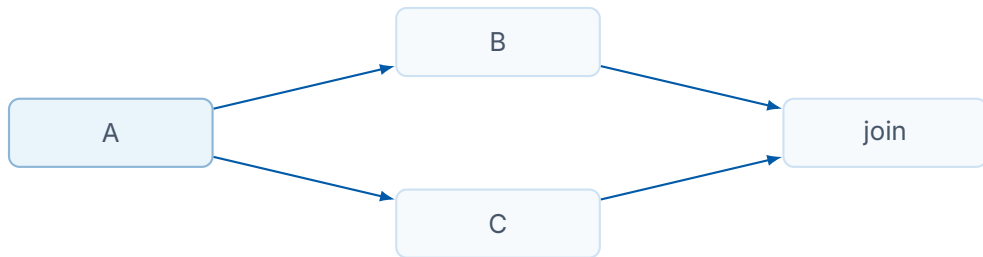
In a simplified sequential chain,

$$T_{e2e} \approx T_A + L_{A,B} + T_B + L_{B,A}.$$

Moving B across a boundary changes the critical path even if the logical computation is unchanged.

$L_{A,B}$ denotes communication delay in this schematic model; it is not assumed symmetric.

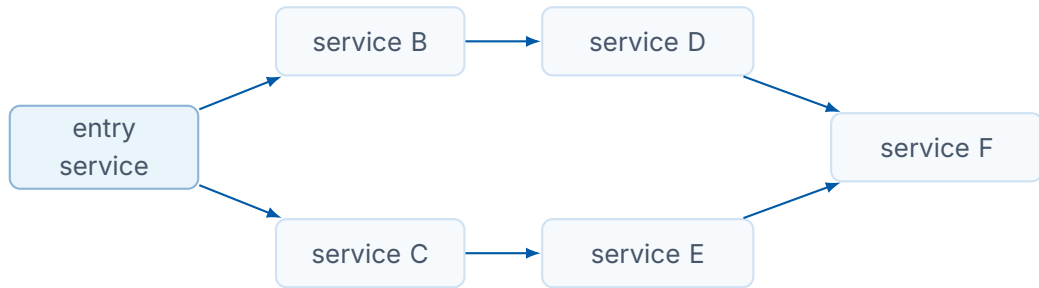
Parallel dependencies behave differently



In the idealized fork/join case,

$$T_{\text{join}} \approx T_A + \max(T_B, T_C).$$

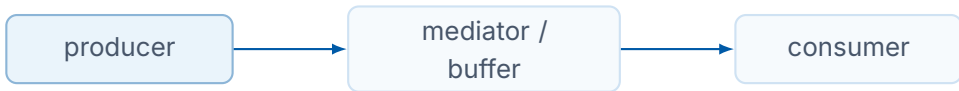
Decomposition changes dependencies -- it does not remove them



Smaller components may improve ownership and independent evolution, but remote dependencies can form **deep interaction chains**.

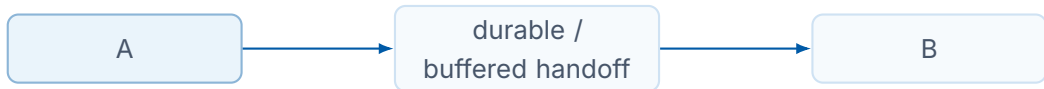
A decomposition decision changes the dependency graph on which latency, failures and coordination propagate.

An intermediary changes the coupling



Routing or buffering can change which component must wait for which other component.

Asynchrony changes the temporal dependency



A may no longer need B to complete before A can make progress.

This does not make the dependency disappear; it changes its timing and state requirements.

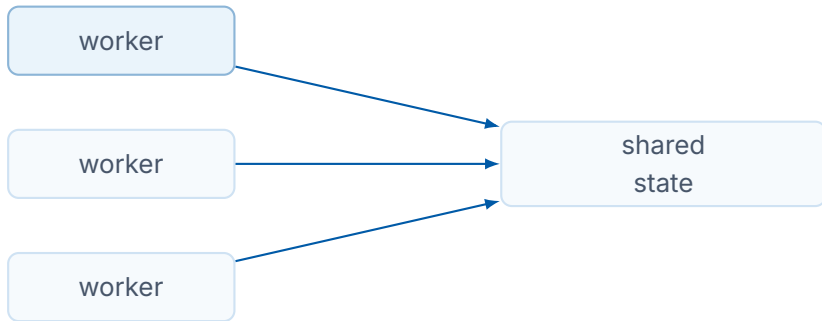
A recurring question

Who has to wait for whom?

6. Architecture changes where constraints appear

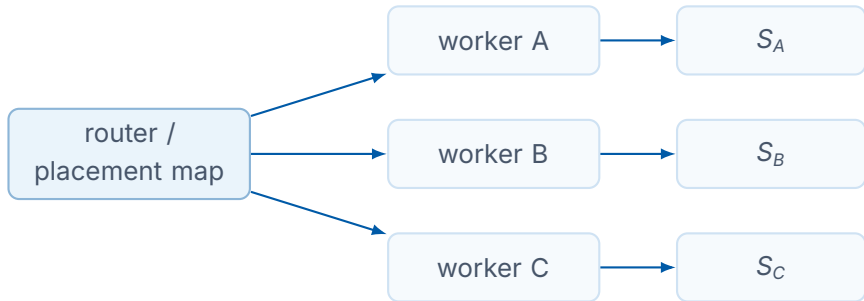
The goal is not to eliminate dependencies, but to understand where they move

Architecture A: replicated computation, shared state



Simple request distribution; common state dependency remains.

Architecture B: ownership and localized state

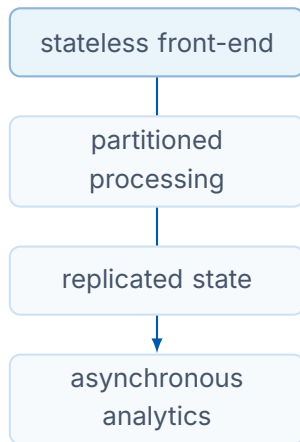


State locality improves; routing, ownership and movement become explicit responsibilities.

A central architectural lesson

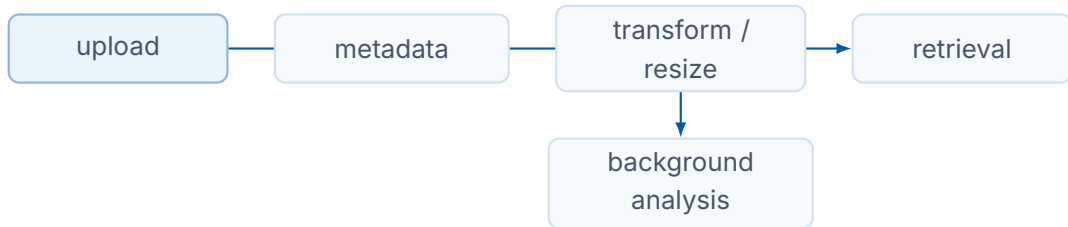
**Scalability constraints
move when architecture
changes; they rarely just
disappear.**

Real systems compose different structures



Different subsystems can legitimately have different state, control and interaction structures.

Worked example: an online photo-processing service



Where should bytes live? Where should transformations execute? Which steps must wait? Who decides placement?

7. Wrap-up

A compact architectural lens for the rest of the course

Four questions to keep asking

Where are the
boundaries?

Where does
state live?

Where are
decisions made?

Who interacts –
and who waits?

What the partitioning model gave us



The theory matters here because it disciplines architectural reasoning about locality, scale and organization – not because we want to solve hypergraph partitioning in this course.

What architecture does not tell us yet

We can now identify where costly dependencies may emerge. We still need to understand:

- how communication cost grows;
- how contention and serialization arise;
- how skew and imbalance destroy ideal scaling;
- how bottlenecks migrate when one tier is improved.

Sources and Limits of Scalability

Communication, contention, coordination, imbalance, skew, serialization and bottleneck migration.

References

Primary sources used in this lecture

References I

- D. L. Parnas. *On the Criteria To Be Used in Decomposing Systems into Modules*. Communications of the ACM, 15(12), 1972.
- M. Stonebraker. *The Case for Shared Nothing*. Database Engineering Bulletin, 9(1), 1986.
- G. Karypis and V. Kumar. *A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs*. SIAM Journal on Scientific Computing, 20(1), 1998.
- M. van Steen and A. S. Tanenbaum. *A brief introduction to distributed systems*. Computing, 98, 2016.

References II

- Ü. V. Çatalyürek and C. Aykanat. *Hypergraph-partitioning-based decomposition for parallel sparse-matrix vector multiplication*. IEEE TPDS, 10(7), 1999.
- B. Hendrickson and T. G. Kolda. *Graph partitioning models for parallel computing*. Parallel Computing, 26(12), 2000.
- A. S. Tanenbaum and M. van Steen. *Distributed Systems*. 4th edition, 2024.